

Chapter 16: Hierarchical Episodic Memory, MemGPT Architectures, and GraphRAG

Preceding Bridge: In [Chapter 06: Memory, Guardrails, Evaluation & Gateways](#), you learned basic conversation sliding windows and vector memory stores. In this chapter, we elevate memory to enterprise scale: how long-running autonomous agents remember interactions across months without overflowing context limits, using **Hierarchical Episodic Memory (MemGPT / Letta)**, **Recursive Summarization (RAPTOR)**, and **Knowledge Graph Augmented Retrieval (GraphRAG)**.

1. Plain-English Jargon Demystifier

Technical Term	Plain English Translation	Real-World Metaphor
Working Memory (RAM)	The immediate context window currently visible to the LLM (e.g., active tokens).	The physical desk space you have to work on right now.
Episodic Memory	A time-ordered log of past events and conversations ("What happened on Tuesday?").	A personal diary with date stamps for every entry.
Semantic Memory	Extracted factual knowledge independent of when it occurred ("Alice is allergic to peanuts").	An encyclopedia entry or flashcard.
Core Memory (Persona)	Read/write slots in the system prompt where the agent records vital facts about the user and itself.	Sticky notes pinned directly to the edge of the computer monitor.
Archival Memory	High-capacity vector and document storage queried via explicit tool calls.	Filing cabinets in the basement searched by filing code.
RAPTOR Trees	Recursive summary trees grouping similar text chunks into multi-layer summaries.	A book with chapter summaries, section summaries, and a 1-page executive summary.
GraphRAG	Extracting entities and relationships into a graph, clustering them into communities, and summarizing whole clusters.	An intelligence agency org-chart connecting persons of interest and their joint organizations.

2. Spoon-Fed Mental Model: The OS Virtual Memory Architecture

How can an operating system run a 100 GB video editing program on a machine with only 16 GB of physical RAM? Through **Virtual Memory and Paging**:

- Active instructions live in **L1/L2 Cache and RAM** (blazing fast, limited space).
- Inactive data is paged out to the **NVMe SSD / Swap file** (huge capacity, slower access).
- When a missing page is requested, a **Page Fault** triggers the OS to fetch it from disk into RAM.

MemGPT (Letta) applies the exact same operating system architecture to LLMs:

The LLM manages its own memory via tool calls:

- `core_memory_append(section, content)` : Updates sticky notes.
- `archival_memory_insert(content)` : Writes long-term knowledge to vector disk.
- `archival_memory_search(query)` : Performs a "page fault" to load past knowledge into context.

3. Microsoft GraphRAG & RAPTOR Trees

Why Flat Vector Search Fails for Global Syntheses

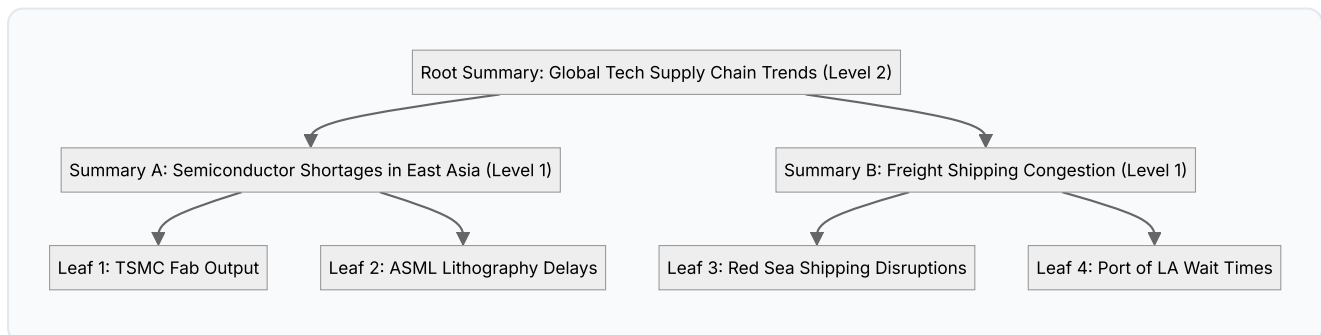
Suppose you feed 1,000 earnings call transcripts into a vector database and ask:

"What were the top 3 global supply chain bottlenecks across all tech companies this quarter?"

Naive RAG completely fails! Every vector chunk talks about one specific factory or port. Cosine similarity only retrieves 5 arbitrary fragments. It has no way to synthesize a bird's-eye view across 1,000 files!

Solution 1: RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval)

1. Split documents into small leaf chunks (L_0).
2. Cluster similar chunks using Gaussian Mixture Models on embedding vectors.
3. Use an LLM to generate a summary for each cluster (L_1).
4. Recursively cluster and summarize the summaries ($L_2 \rightarrow L_{root}$).
5. **Retrieval:** If the query is broad and thematic, search high-level tree nodes (L_2); if the query is detailed, search leaf nodes (L_0).



Solution 2: GraphRAG (Entity-Relationship Knowledge Graphs)

1. **Extraction:** LLM extracts entities (`Company` , `Person` , `Location` , `Technology`) and relationships (`SUPPLIES_TO` , `INVESTS_IN`).
2. **Community Detection:** Uses the **Leiden Algorithm** to detect hierarchical communities of densely connected entities in the graph.
3. **Community Reports:** LLMs summarize each community at multiple granularity levels.
4. **Global Query Answering:** The system answers high-level questions by mapping-reducing over community summaries rather than individual raw chunks.

4. Junior vs Staff Implementation: Agent Memory Systems

5. Complete Runnable Implementation: MemGPT Memory Engine

Here is a pure-Python, zero-dependency implementation of an OS-style hierarchical memory manager:

6. Chapter Milestone Check

Verify your understanding before moving forward:

1. **How does MemGPT prevent context window exhaustion in continuous running agents?**
 - *Answer:* It treats the context window as RAM and separates it from external storage (disk). When the working memory reaches capacity, older messages are automatically summarized, paged out to archival storage, and retrieved only when the agent issues a search tool call.
2. **What is the key advantage of RAPTOR's tree-structured retrieval over standard flat chunking?**
 - *Answer:* Flat chunking cannot answer broad, thematic questions spanning an entire library. RAPTOR builds recursive cluster summaries, allowing queries to be answered at varying levels of semantic abstraction.
3. **In GraphRAG, what does the Leiden community detection algorithm accomplish?**

- *Answer:* It partitions densely connected clusters of entities (e.g., all people and projects within a single department) so the system can generate structured, high-level summaries of entire interconnected domains.

Further Reading

- [GraphRAG paper](#)
 - [Microsoft GraphRAG documentation](#)
 - [MemGPT paper](#)
-

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▶ 1. What does episodic memory store?
- ▶ 2. When does GraphRAG help most?
- ▶ 3. What is the main cost of GraphRAG?
- ▶ 4. Why timestamp and attribute memories?